For this reason some teams choose to note on each task who is the person most likely to work on that task. Often this is an unnecessary and potentially wasteful step because unexpected events will occur during the sprint and tasks will need to be reassigned. Also, assigning tasks to individuals might be harmful if doing so reduces team ownership of the tasks. A better strategy (as I will discuss in Chapter 20) is to let team members select the work in an opportunistic, just-in-time fashion during the sprint.

Refine the Sprint Goal

The sprint goal summarizes the business purpose and value of the sprint. The product owner should come to sprint planning with an initial sprint goal. That initial goal, however, can be refined during the course of sprint planning as the sprint-planning participants work together to determine what can realistically be delivered.

Finalize the Commitment

At the completion of sprint planning the development team finalizes its commitment to the business value it will deliver by the end of the sprint. The sprint goal and the selected product backlog items embody that commitment.

As I mentioned in Chapter 2, some people prefer to use the term *forecast* to describe the business value that the development team believes it can produce by the end of the sprint. I prefer and use the term *commitment*. Regardless of which term you prefer, the approaches to sprint planning I describe in this chapter are the same.

The nuanced differences between these terms might affect only the scope of what the development team determines it can deliver and how the Scrum team deals with new information that arrives during sprint execution (see Chapter 4 for a discussion of changes during a sprint).

Closing

In this chapter I expanded the description of sprint planning by discussing when sprint planning takes place and who is involved. I discussed two different approaches to sprint planning. In the first approach the team selects a set of product backlog items and then acquires confidence that it can indeed deliver the full set. The second approach intermingles selecting a product backlog item with acquiring confidence that the item can be added to the incrementally growing commitment. I also explained two different ways to determine a development team's capacity to complete work. In the next chapter I will discuss details of how sprints are executed once they have been planned.

Chapter 20
Sprint Execution

Sprint execution is the work the Scrum team performs to meet the sprint goal. In this chapter I focus on the principles and techniques that guide how the Scrum team plans, manages, performs, and communicates during sprint execution.

Overview

Sprint execution is like a mini project unto itself—all of the work necessary to deliver a potentially shippable product increment is performed.

Timing

Sprint execution accounts for the majority of time during a sprint. It begins after sprint planning and ends when the sprint review starts (see Figure 20.1).

On a two-week-long sprint, sprint execution might account for about eight out of the ten days.

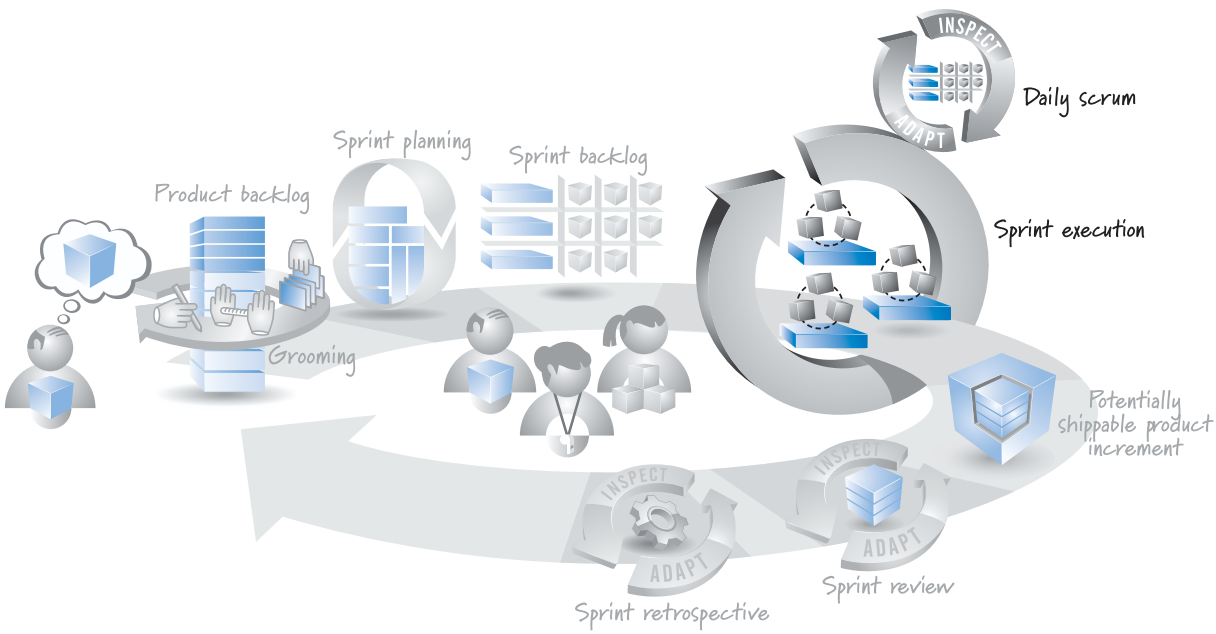


FIGURE 20.1 When sprint execution happens

Participants

During sprint execution the development team members self-organize and determine the best way to meet the goal established during sprint planning.

The ScrumMaster participates as the coach, facilitator, and impediment remover, doing whatever is possible to help the team be successful. The ScrumMaster doesn't assign work to the team or tell the team how to do the work. A self-organizing team figures these things out for itself.

The product owner must be available during sprint execution to answer clarifying questions, to review intermediate work and provide feedback to the team, to discuss adjustments to the sprint goal if conditions warrant, and to verify that the acceptance criteria of product backlog items have been met.

Process

Figure 20.2 illustrates the sprint execution activity.

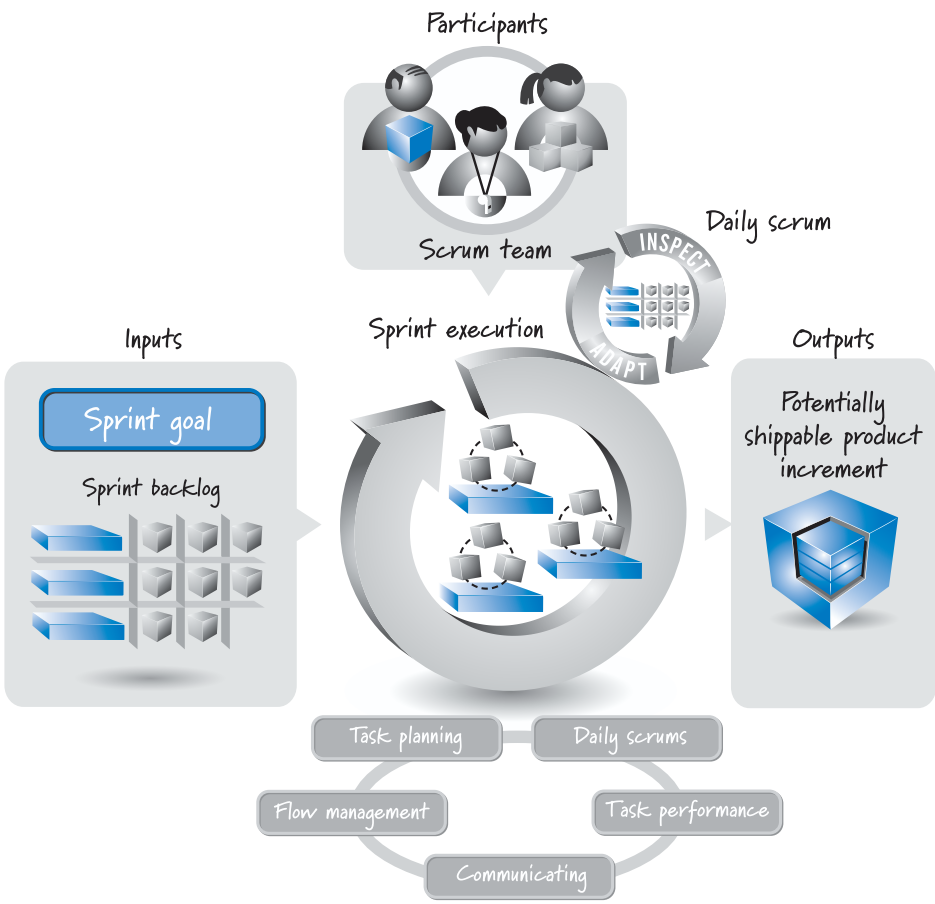


FIGURE 20.2 Sprint execution activity

The inputs to sprint execution are the sprint goal and the sprint backlog that were generated during sprint planning. The output from sprint execution is a potentially shippable product increment, which is a set of product backlog items completed to a high degree of confidence—where each item meets the Scrum team's agreed-upon definition of done (see Chapter 4). Sprint execution involves planning, managing, performing, and communicating the work necessary to create these working, tested features.

Sprint Execution Planning

During sprint planning the team produces a *plan* for how to achieve the sprint goal. Most teams create a sprint backlog, which typically lists product backlog items and their associated tasks and estimated effort-hours (see Figure 19.6). Although the team probably could create a full task-level sprint execution plan (the equivalent of a project plan for the sprint, perhaps in the format of a Gantt chart), the economics of doing so are hard to justify.

First, it's not clear that a team of five to nine people needs a Gantt chart to dictate who should do the work and when for the next short-duration sprint. Second, even if the team wanted to create a Gantt chart, it would be inaccurate soon after the team begins working. Sprint execution is where the rubber meets the road. A massive influx of learning comes from actually building and testing something. This learning will disrupt even the best-conceived early plan. As a result, the team wastes valuable time putting a plan together, only to waste even more time changing it to reflect the reality of sprint execution.

Of course, some up-front planning is helpful for exposing important task-level dependencies. For example, if we know that a feature we're creating during the sprint must be subjected to a special two-day-long stress test, it would be wise for the team to sequence the work so that this test starts at least two days before the end of sprint execution.

A good principle for sprint execution is to approach task-level planning opportunistically rather than trying to lay out up front a complete plan of how to do the work (Goldberg and Rubin 1995). Allow task planning to occur continuously during sprint execution as the team adapts to the evolving circumstances of the sprint.

Flow Management

It's the team's responsibility to manage the flow of work during sprint execution to meet the sprint goal. It must make decisions such as how much work the team should do in parallel, when work should begin on a specific item, how the task-level work should be organized, what work needs to be done, and who should do the work.

When answering these questions, teams should discard old behaviors, such as trying to keep everyone 100% busy (the consequences of which are described in Chapter 2), believing that work must be done sequentially, and having each person focus on just her part of the solution.

Parallel Work and Swarming

An important part of managing flow is determining how many product backlog items the team should work on in parallel to maximize the value delivered by the end of the sprint. Working on too many items at once contributes to team member multitasking, which in turn increases the time required to complete individual items and likely reduces their quality.

Figure 20.3 shows a simple example that I use in my training classes to illustrate the cost of multitasking.

The goal is to complete two identical tables by writing the letters *a* to *j*, the numbers 1 to 10, and Roman numerals i to x. One table is completed a row at a time, and the other is completed a column at a time. The row-at-time table represents multitasking (do the letter task, then do the number task, then do the Roman numeral task, and then repeat the sequence for the next letter, number, and Roman numeral). The column-at-a-time table represents single tasking.

The typical results shown in Figure 20.3 are that most people complete the column-at-a-time table in about half the time of the row-at-a-time table. Give it a try

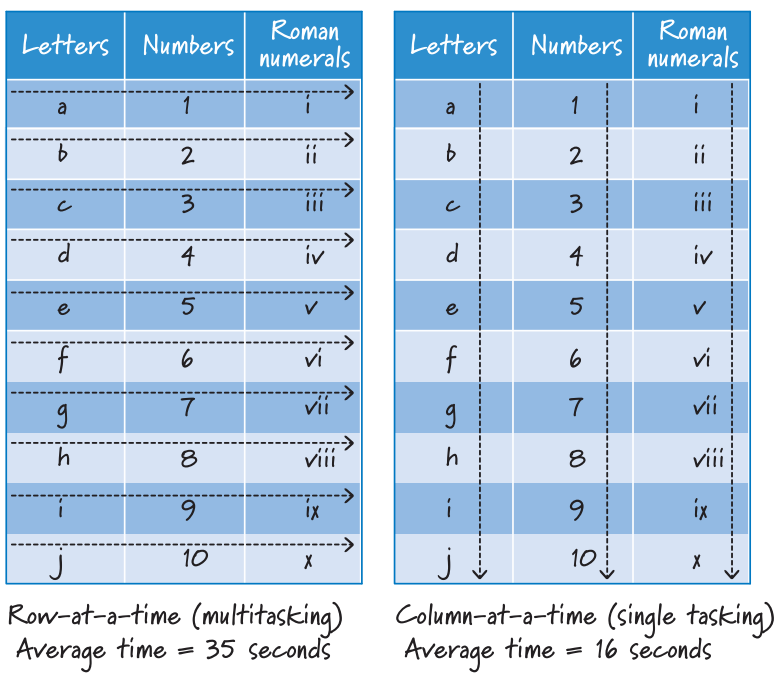


FIGURE 20.3 Cost of multitasking

and time yourself, and you'll see! Also, if people make any errors, they make them when completing the row-at-a-time table. So, even for simple multitasking the overhead can be quite high. Imagine the waste involved when multitasking on complex project work.

Just as working on too many items at the same time is wasteful, working on too few items at a time is also wasteful. It leads to underutilization of team member skills and capacity, resulting in less work being completed and less value being delivered.

To find the proper balance, I recommend that teams work on the number of items that leverages, but does not overburden, the T-shaped skills (see Chapter 11) and available capacity (see Chapter 19) of the team members. The goal is to reduce the time required to complete individual items while maximizing the total value delivered during the sprint (typically the number of items completed during the sprint).

A term frequently used to describe this approach is **swarming**, where team members with available capacity gather to work on an item to finish what has already been started before moving ahead to start work on new items. Teams with a Musketeer attitude and some degree of T-shaped skills swarm. Teams that still think in terms of individual roles wind up with some members far ahead and others who are mired in unfinished work. A classic individual-role-focused thought is "The testers might still have 'their' work to finish up, but I'm finished coding this feature, so I'm off to start coding the next one." In a team that swarms, people would understand that it is typically better to stay focused and help get the testing done instead of running ahead to start working on new features.

Some people mistakenly believe that swarming is a strategy to ensure that team members are 100% busy. This is not the goal of swarming. If we wanted to ensure that people were 100% busy, we would just start working on all product backlog items at the same time! Why don't we do that? Because the extensive multitasking required to make that happen would ultimately slow the flow of completed items. Swarming, on the other hand, helps the team remain goal focused instead of task focused, which means it gets more things done, faster.

While swarming favors working on fewer items concurrently, it doesn't necessarily mean working on only one product backlog item at a time. One item at a time might be correct in a given context, but just saying that all team members should collectively focus on a single item at a time is potentially dangerous. A different number of items might be appropriate when we consider the actual work that needs to be done, the skills of the team members, and other conditions that exist at the time a decision to start or not start working on another item needs to be made.

Another dangerous approach would be to apply waterfall thinking at the sprint level and treat sprint execution like a mini waterfall project. Using this approach, we would start working on all product backlog items at the same time. First we would analyze all of the items to be worked on this sprint, then design them all, then code them all, and then, finally, test them all (see Figure 20.4).

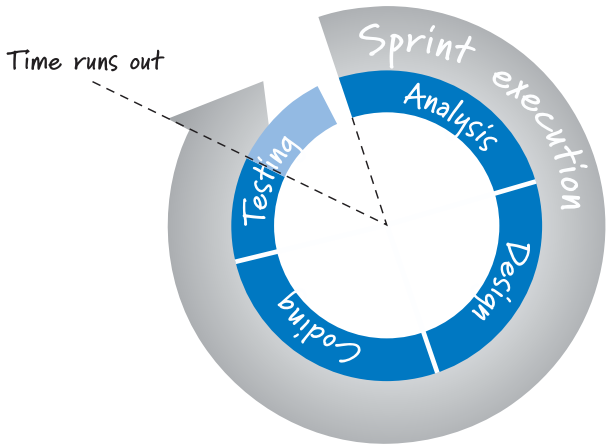


FIGURE 20.4 Mini waterfall during sprint execution—a bad idea

Although this approach may seem logical, it is very risky. What if the team runs out of time and doesn't finish all of the testing? Do we have a potentially shippable product increment? No; a reasonable definition of done would never allow untested features to be called done. By using a mini waterfall strategy, we could end up with 90% of each feature complete, but no feature 100% done. The product owner gets no economic value from partially done work.

Which Work to Start

Assuming that not all product backlog items are started simultaneously, at some point the team needs to determine which product backlog item to work on next.

The simplest way to select the next product backlog item is to choose the next-highest-priority item as specified by the product owner (via the item's position in the product backlog). This approach has the obvious advantage of ensuring that any items not completed during the sprint must be of lower priority than the ones that are completed.

Unfortunately, the simplest approach won't always work because technical dependencies or skills capacity constraints might dictate that items be selected in a different order. The development team needs the ability to opportunistically make this selection as it sees fit.

How to Organize Task Work

Once the development team decides to start working on a product backlog item, it must determine how to perform the task-level work on that item. If we apply waterfall thinking at the level of a single product backlog item, we would analyze the item, design it, code it, and then test it.

Believing there is a single, predetermined, logical ordering to the work (for example, you have to build it before you can do any testing) blinds the team to the opportunity to do things in a different and perhaps more efficient way. For example, I frequently hear new teams say something like "What will our testers be doing early in the sprint while they are waiting for features to be ready for testing?" Typically I respond by saying that for teams doing **test-first development**, where tests are written before the development is performed, the "testers" are the *first* to work on a feature (Crispin and Gregory 2009)!

Traditional role-based thinking plagues many teams. What we need instead is value-delivery-focused thinking, where the team members opportunistically organize the tasks and who will work on them. In doing so, they minimize the amount of time that work sits idle and reduce the size and frequency at which team members must "hand off" work to one another. This might mean, for example, that two people pair up on the first day of sprint execution and work in a highly interleaved fashion, with rapid cycles of test creation, code creation, test execution, and test and code refinement, and then repeat this cycle. This approach keeps work flowing (no blocked work), supports very fast feedback so that issues are identified and resolved quickly, and enables team members with T-shaped skills to swarm on an item to get it done.

What Work Needs to Be Done?

What task-level work does the team perform to complete a product backlog item? Ultimately the team decides. Product owners and managers must trust that the team members are responsible professionals who have a vested interest in doing great work. As such, they need to empower these individuals to do the necessary work to create innovative solutions in an economically sensible way.

Of course, product owners and managers do have influential input to what task-level work gets done. First, the product owner ensures that the scope of a feature and its acceptance criteria are defined (part of the definition of ready described in Chapter 6), both of which provide boundaries for the task-level work.

Product owners and managers also provide business-facing requirements for the definition of done. For example, if the business requires that the features developed in each sprint be released to the end customer at the conclusion of the sprint, that decision influences the task-level work the team will perform (there is more work involved with getting features live on production servers than there is in getting them built and tested).

Overall, the product owner must work with the team to ensure that technical decisions with important business consequences are made in an economically sensible way. Some of these decisions are embedded in the more technically oriented aspects of the definition of done. For example, the Scrum team may collectively decide that having automated regression tests (which have an economic cost and

benefit) is important and in this way influence task-level work (to create and run automated tests).

Other decisions are feature specific. There is often a degree of flexibility regarding how much effort a team should exert on a feature. For example, enhancing or polishing a feature might be technically appealing but simply not worth the extra cost to the product owner at this time or ever. Conversely, cutting corners on a design or shortchanging where, how, or when we do testing also has economic consequences that must be considered (see the discussion of technical debt in Chapter 8). The team is expected to work with the product owner to discuss these trade-offs and make economically sensible choices.

Who Does the Work?

Who should work on each task? An obvious answer is the person best able to quickly and correctly get it done. What if that person is unavailable? Perhaps she is already working on another, more important task, or maybe she is out sick and the task needs to get done immediately.

There are a number of factors that can and should influence who will work on a task; it's the collective responsibility of the team members to consider those factors and make a good choice.

When team members have T-shaped skills, several people on the team have the ability to work on each task. When some skills overlap among team members, the team can swarm people to the tasks that are inhibiting the flow of a product backlog item through sprint execution, making the team more efficient.

Daily Scrum

The daily scrum is a critical, daily inspect-and-adapt activity to help the team achieve faster, more flexible flow toward the solution. As I discussed in Chapter 2, the daily scrum is a 15-minute, timeboxed activity that takes place once every 24 hours. The daily scrum serves as an inspection, synchronization, and daily adaptive planning activity that helps a self-organizing team do its job better.

The goal of the daily scrum is for people who are focused on meeting the sprint goal to get together and share the big picture of what is happening so that they can collectively understand how much to work on, which items to start working on, and how to best organize the work among the team members. The daily scrum also helps avoid waiting. If there is an issue that is blocking flow, the team would never have to wait more than a day to discuss it. Imagine if the team members got together only once a week—they would deny themselves the benefits of fast feedback (see Chapter 3). Overall the daily scrum is essential for flow management.

Task Performance—Technical Practices

Development team members are expected to be technically good at what they do. I'm not saying that you need a team of superstars to use Scrum. However, working in short, timeboxed iterations where there is an expectation of delivering potentially shippable product increments does exert pressure on teams to get the job done with good control over technical debt. If team members lack appropriate technical skills, they will likely fail to achieve the level of agility needed to deliver long-term, sustainable business value.

If you are using Scrum to develop software, team members need to be skilled in good technical practices for developing software. I'm not referring to esoteric skills but instead to skills that have been in use for decades and are essential to being successful with Scrum or arguably any software development approach—for example, continuous integration, automated testing, refactoring, test-driven development, and so on. Today the agile community refers to many of these technical practices as Extreme Programming (Beck and Andres 2004), but most are practices that predate that label (see Figure 20.5 for a subset of the Extreme Programming technical practices).

As an example, consider automated testing, which is necessary to support several of the practices in Figure 20.5. Development teams that don't focus on automating their tests will quickly start to slow down and take ever-increasing risks. At some point, it could take all of the sprint execution time just to manually rerun the regression tests for previously developed features. In such cases, the team might choose not

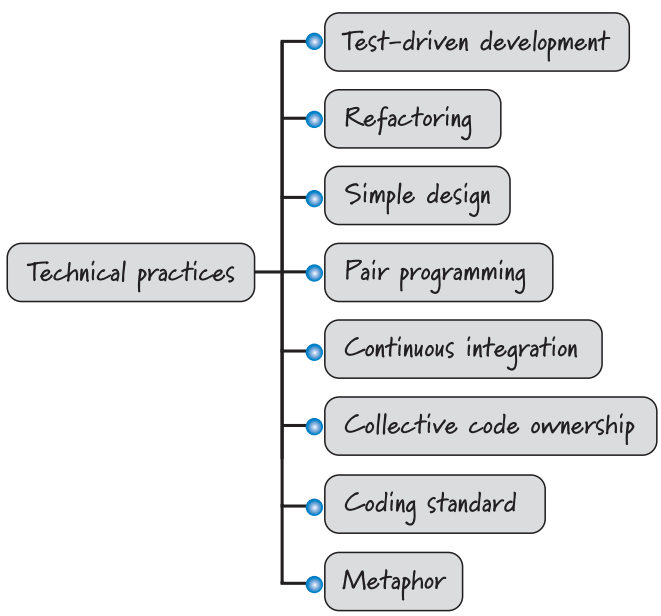


FIGURE 20.5 Subset of Extreme Programming technical practices

to rerun all of the manual tests each sprint, which could allow defects to propagate forward, adding to the system's technical debt (increased risk). You won't be agile for very long if you don't start automating your tests.

Similar arguments can be made for other core technical practices. Most teams achieve the long-term benefits of Scrum only if they also embrace strong technical practices when performing task-level work.

Communicating

One of the benefits of working in short timeboxes with small teams is that you don't need complex charts and reports to communicate progress! Although any highly visible way of communicating progress can be used, most teams use a combination of a task board and a burndown and/or burnup chart as their principal **information radiator**.

Task Board

The **task board** is a simple but powerful way to communicate sprint progress at a glance. Formally, the task board shows the evolving state of the sprint backlog over time (see Figure 20.6).

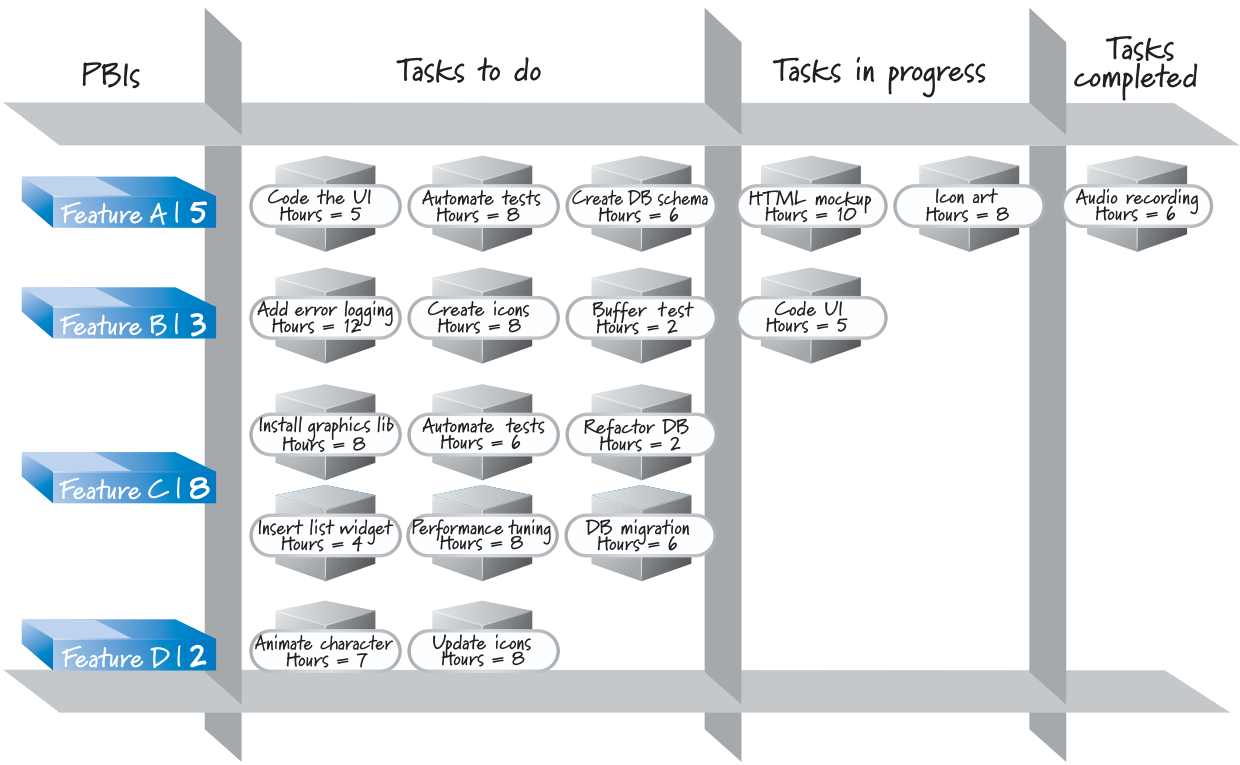


FIGURE 20.6 Example task board

On this task board each product backlog item planned to be worked on during the sprint is shown with the set of tasks necessary to get the item done. All tasks initially start off in the "to do" column. Once the team determines that it is appropriate to work on an item, team members start selecting tasks in the "to do" column for the item and move them into the "in progress" column to indicate that work on those tasks has begun. When a task is completed, it is moved to the "completed" column.

Of course, Figure 20.6 is just an example of how a task board might be structured. A team may choose to put other columns on its task board if it thinks that visualizing the flow of work through other states is helpful. In fact, an alternative agile approach called Kanban (Anderson 2010) uses just such a detailed board to visualize the flow of work through its various stages.

Sprint Burndown Chart

Each day during sprint execution team members update the estimate of how much effort remains for each uncompleted task. We could create a table to visualize this data. Table 20.1 shows an example of a 15-day sprint that initially has 30 tasks (not all of the days and tasks are shown in the table).

In Table 20.1 the number of hours remaining for each task follows the general trend of being smaller each day during the sprint—because tasks are being worked on and completed. If a task hasn't yet been started (it is still in the task board "to do" column), the size of the task might appear the same from day to day until the task is started. Of course, a task might turn out to be larger than expected, and if so, its size may actually increase day over day (see Table 20.1, task 4, days 4 and 5) or remain the same size even after the team has started working it (see Table 20.1 task 1, days 2 and 3)—either

TABLE 20.1 Sprint Backlog with Estimated Effort Remaining Each Day

Tasks	D1	D2	D3	D4	D5	D6	D7	D8	D9	. . .	D15
Task 1	8	4	4	2							
Task 2	12	8	16	14	9	6	2				
Task 3	5	5	3	3	1						
Task 4	7	7	7	5	10	6	3	1			
Task 5	3	3	3	3	3	3	3				
Task 6	14	14	14	14	14	14	14	8	4		
Task 7						8	6	4	2		
Tasks 8–30	151	139	143	134	118	99	89	101	84		0
Total	200	180	190	175	155	130	115	113	90		0

because no work took place on the task the previous day, or work did take place the previous day but the estimated effort remaining is the same.

New tasks related to the committed product backlog items can also be added to the sprint backlog at any time. For example, on day 6 in Table 20.1 the team discovered that task 7 was missing, so it added it. There is no reason to avoid adding a task to the sprint backlog. It represents real work that the team must do to complete a product backlog item that the team agreed to get done. Permitting unforeseen tasks to be added to the sprint backlog is not a loophole for introducing new work into the sprint. It simply acknowledges that during sprint planning we may not be able to fully define the complete set of tasks needed to design, build, integrate, and test the committed product backlog items. As our understanding of the work improves by doing it, we can and should adjust the sprint backlog.

If we plot the row labeled "Total" in Table 20.1, which is the sum of the remaining effort-hours across all uncompleted tasks on a given day, on a graph, we get another of the Scrum artifacts for communicating progress—the sprint burndown chart (see Figure 20.7).

In Chapter 18 I discussed release burndown charts, where the vertical axis numbers are either in story points or ideal days and the horizontal axis numbers are in sprints (see Figure 18.11). In sprint burndown charts the vertical axis numbers are the estimated effort-hours remaining, and the horizontal axis numbers are days within a sprint. Figure 20.7 shows that we have 200 estimated effort-hours remaining on the first day of the sprint and zero effort-hours remaining on day 15 (the last day of a three-week-long sprint). Each day we update this chart to show the total estimated effort remaining across all of the uncompleted tasks.

Like release burndown charts, sprint burndown charts are useful for tracking progress and can also be used as a leading indicator to predict when work will be

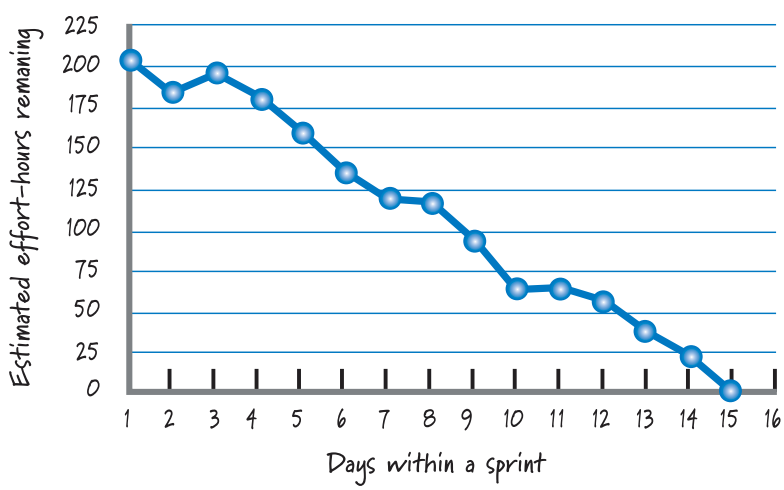


FIGURE 20.7 Sprint burndown chart

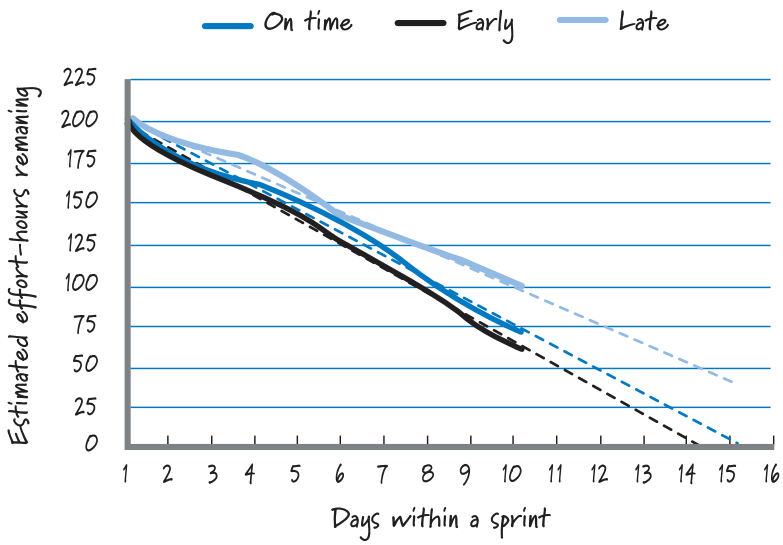


FIGURE 20.8 Sprint burndown chart with trend lines

completed. At any point in time we could compute a trend line based on historical data and use that trend line to see when we are likely to finish if the current pace and scope remain constant (see Figure 20.8).

In this figure, three different burndown lines are superimposed to illustrate distinct situations. When the trend line intersects the horizontal axis close to the end of the sprint duration, we can infer that we're in reasonable shape ("On time"). When it lands significantly to the left, we should probably take a look to see if we can safely take on additional work ("Early"). But when it lands significantly to the right ("Late"), that raises a flag that we're not proceeding at the expected pace or that we've taken on too much work (or both!). When that happens, we should dig deeper to see what's behind the data and what, if anything, needs to be done. By projecting the trend lines, we have another important set of data that adds to our knowledge of how we are managing flow within our sprint.

The sprint backlog and the sprint burndown charts always use estimated effort *remaining*. They do not capture actual effort expended. In Scrum there is no specific need to capture the *actuals*; however, your organization might choose to do so for non-Scrum reasons such as cost accounting or tax purposes.

Sprint Burnup Chart

Analogous to how a release burnup chart is an alternative way of visualizing progress through a release, a sprint burnup chart is an alternative way to visualize progress through a sprint. Both represent the amount of work completed toward achieving a goal, the release goal in one case and the sprint goal in the other.

Figure 20.9 shows an example sprint burnup chart.

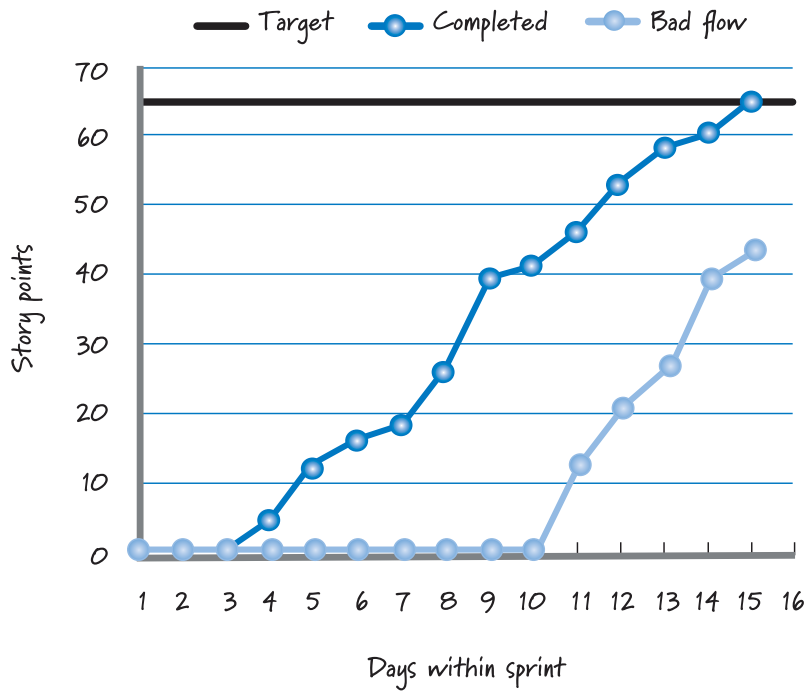


FIGURE 20.9 Sprint burnup chart

In sprint burnup charts the work can be represented in either effort-hours (as in the sprint burndown chart) or in story points (as shown in Figure 20.9). Many people prefer to use story points in their burnup charts, because at the end of the sprint the only thing that really matters to the Scrum team is business-valuable work that was completed during the sprint, and that is measured in story points (or ideal days), not task hours.

Also, if we measure story points of completed product backlog items, at a glance we can get a good feel for how the work is flowing and how the team is completing product backlog items through the sprint. To illustrate this point a third line (labeled "Bad flow") is included on the sprint burnup chart in Figure 20.9 (normally this line would not be on the chart; it is added in this example for comparison purposes). The "Bad flow" line illustrates what the burnup chart might look like if the team starts too many product backlog items at the same time, delays completion of items until later in the sprint, fails to meet the sprint target because of the reduced velocity of doing too much work in parallel, works on product backlog items that are large and therefore take a long time to finish, or takes other actions that result in bad flow.

Closing

In this chapter I discussed sprint execution, which accounts for the majority of the time during a sprint. I emphasized that sprint execution is not guided by a complete up-front plan that specifies what work will be done, when it will be done, and who will

do it. Rather, sprint execution is performed opportunistically, leveraging the skills of the team, feedback from work already completed, and the evolving, unpredictable circumstances of the sprint. This doesn't mean that sprint execution is chaotic, but rather that it is guided by the application of good flow management principles, which determine how much work to do in parallel, which work to start, how to organize that work, who will do the work, and how much effort to invest in the work. In this context I discussed the value of the daily scrum meeting as an important activity in flow management. I also mentioned the importance of good technical practices in achieving high levels of agility. I concluded by discussing the various ways that the Scrum team can visually communicate sprint progress through the task board, sprint burndown chart, and sprint burnup chart. In the next chapter I will discuss the sprint review activity that naturally follows sprint execution.

This page intentionally left blank

Chapter 21
Sprint Review

Near the end of the sprint, the team conducts two important inspect-and-adapt activities: the sprint review and the sprint retrospective. The sprint review focuses on the product itself. The sprint retrospective, on the other hand, looks at the *process* the team is using to build the product.

In this chapter I describe the sprint review—its purpose, its participants, and the work required to make it happen. I conclude by addressing a few common sprint review issues.

Overview

During sprint planning we plan the work. During sprint execution we do the work. During sprint review we inspect (and adapt) the result of the work (the potentially shippable product increment). The sprint review occurs near the end of each sprint cycle, just after sprint execution and just before (or occasionally after) the sprint retrospective (see Figure 21.1).

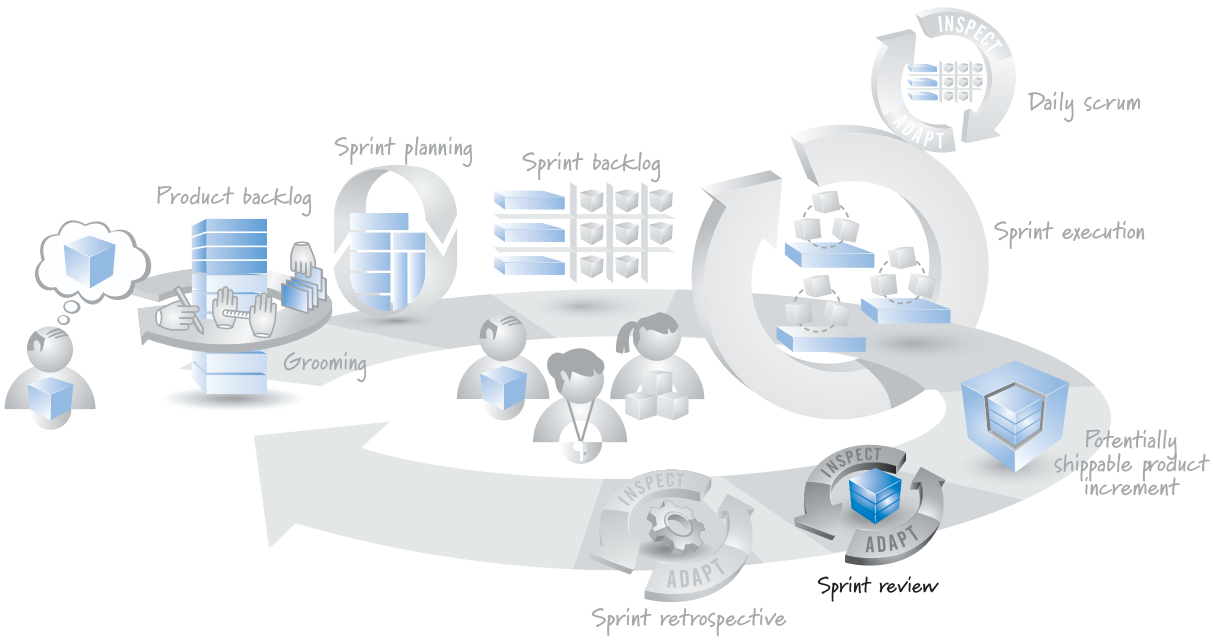


FIGURE 21.1 When the sprint review happens

The sprint review gives everyone with input to the product development effort an opportunity to inspect and adapt what has been built so far. The sprint review provides a transparent look at the current state of the product, including any inconvenient truths. It is the time to ask questions, make observations or suggestions, and have discussions about how to best move forward given current realities.

Because it helps ensure that the organization is creating a successful product, the sprint review is one of the most important learning loops in the Scrum framework. And, because sprints are short, this loop is a quick one, which allows for frequent course corrections to keep the product development moving in the right direction. If, instead, we were to defer this feedback until much later and assume that everything is going according to some baseline plan, we likely would get what many are accustomed to—surprise, disappointment, and frustration.

Participants

The sprint review provides an important opportunity for the Scrum team to get feedback from people who typically are not available on a daily basis during sprint execution. For these individuals, the sprint review is their first opportunity to see and discuss the work that was produced during the sprint. The sprint review, therefore, should be attended by all interested parties, who can come from a number of different sources, as summarized in Table 21.1.

TABLE 21.1 Sprint Review Attendee Sources

Source	Description
Scrum team	The product owner, ScrumMaster, and development team should all be present so that they can all hear the same feedback and be able to answer questions regarding the sprint and the product increment.
Internal stakeholders	Business owners, executives, and managers should see the progress firsthand so that they can suggest course corrections. For internal product development, internal users, subject matter experts, and the operations manager of the business function to which the product relates should attend.
Other internal teams	Sales, marketing, support, legal, compliance, and other Scrum and non-Scrum development teams might want to attend sprint reviews to provide area-specific feedback or to sync their own groups' work with the Scrum team.
External stakeholders	External customers, users, and partners can provide valuable feedback to the Scrum team and other attendees.

All Scrum team members (product owner, ScrumMaster, and development team) should be present at every sprint review so that they can describe what has been accomplished, answer questions, and enjoy the benefits of firsthand feedback.

Internal stakeholders, such as business-area owners (who may be paying for the system being built), executive management, and resource and other managers, should also attend. Their feedback is essential to ensuring that the team is progressing toward an economically sensible outcome. In addition, sprint reviews provide a convenient opportunity to learn the status of the product development effort. Also, for internal development efforts, the users will be internal to the organization; a representative sample of these users should attend along with subject matter experts who are an excellent source of feedback on what has been built.

Others in your organization might want to attend as well. Salespeople and marketing specialists frequently sit in. They can be an excellent source of feedback on whether the product is converging on a marketplace success. Other groups, such as support, legal, and compliance, might also come to sprint reviews to stay abreast of the team's progress, to provide timely input to the team, and to better gauge when to start their own related work.

Other internal development teams on related development efforts might send representatives so that they can ascertain where the product is headed and provide any relevant input on what they are doing and how it might impact the current development effort.

It's a good idea to at least periodically include external stakeholders, such as actual customers or users of what the team is building. With them in the room, the team can get direct feedback instead of indirect (or proxy) feedback via internal stakeholders. It may not make sense to have external stakeholders at every review, especially if we know that a particular review might involve some intense internal discussions that are best conducted with internal stakeholders only. If we do choose to include external stakeholders, unless there is just a single stakeholder, some consideration should be given to which of the potentially many customers or users we should invite. Common sense as well as sensitivity to the desires and personalities of specific people should be good guides on whom to invite.

Prework

Although the sprint review is an informal activity, the Scrum team has some minimal prework to complete (see Figure 21.2).

This prework includes determining whom to invite, scheduling the sprint review, confirming that the sprint work is done, preparing for the sprint review demonstration, and deciding who will lead the meeting and who will give the demo.

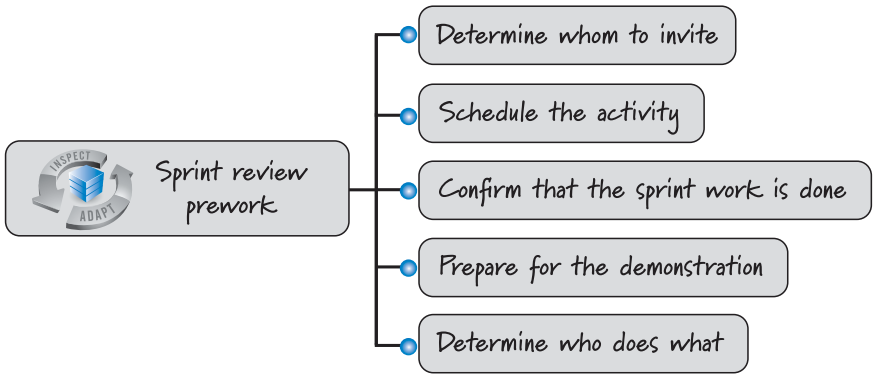


FIGURE 21.2 Sprint review prework

Determine Whom to Invite

The Scrum team first needs to determine who should attend the sprint review on a regular basis. The goal is to get the right set of people into the room to extract the highest possible value. Unless there is a good reason to not invite someone or some group, cast a broad net and let people vote with their feet—if they're interested, they'll walk to the room and attend the meeting.

Occasionally, the team might need to constrain attendance. For instance, the team might need to focus on a certain person or group whose input is essential to reviewing this sprint's work. Or the team might be building a feature for a specific client during this sprint and so cannot invite that client's competitors to the review meeting.

If you suspect these situations might arise, identify a core group that should be invited to every review and then issue a separate invitation to certain groups or clients on a sprint-by-sprint basis.

Schedule the Activity

The sprint review needs to be scheduled (when, where, and how long). Of the four required, recurring Scrum activities (sprint planning, daily scrum, sprint review, and sprint retrospective), the sprint review is the hardest to schedule because it includes many people who are outside of the Scrum team. The other three recurring activities involve only people on the Scrum team and therefore can be scheduled at its convenience alone.

To make scheduling easier, begin by determining when the key stakeholders (the core group I mentioned earlier) would prefer to hold the sprint review—say, Friday afternoons at 2:00 p.m.—and then schedule the rest of the sprint activities around this fixed time. If, as I discussed in Chapter 4, we use consistent-duration sprints (say, every two weeks), we can then schedule all, or at least most, of the sprint review

meetings using a regular cadence (every second Friday at 2:00 p.m.). This has the dual benefit of reducing the administrative burden and costs while increasing attendance.

Sprint reviews vary in duration depending on several factors, including sprint length, team size, and whether multiple teams are participating in the same review. Typically, however, the sprint review does not exceed a four-hour timebox. Many teams have found the one-hour-per-sprint-week rule helpful. In other words, for a two-week sprint the review should take no more than two hours; for a four-week sprint it should take no more than four hours.

Confirm That the Sprint Work Is Done

At the sprint review, the team is allowed to present only completed work—work that meets the agreed-upon definition of done. (See Chapter 4 for more on this topic.) This implies, then, that sometime *before* the sprint review, someone has determined whether or not each backlog item is done; otherwise, how would the Scrum team know which items to present?

Ultimately it is the product owner's responsibility to determine if the work is done or not. As I mentioned in Chapter 9, the product owner should be performing just-in-time reviews of product backlog items as they become available during sprint execution. This way, by the time the sprint review happens, the team knows which items are complete.

Not everyone agrees that the product owner should review the work before the sprint review. Some practitioners contend that the product owner should review and formally accept the work only during the sprint review. They believe that if the product owner is allowed to review the work during the sprint, he might request changes that go beyond clarification—goal-altering changes that will disrupt sprint execution (see Chapter 4).

This is a potential risk, but the benefits of early product owner reviews (fast feedback) far outweigh any downside. Furthermore, if the product owner sees the team's work for the first time at the sprint review meeting, he has seen it too late. Here's why. The product owner must be available during sprint execution to answer questions and clarify product backlog items. While fulfilling these obligations, the product owner should also review the ongoing progress the team is making and provide critical, in-flight feedback that can be acted on in a timely, cost-effective manner. Deferring this feedback until the sprint review would create unnecessary work and likely frustrate the team ("Why didn't you mention that during the sprint, when we could have fixed it easily?"). It also could potentially irritate the stakeholders ("This feature would have been potentially shippable if you had just handled those things during the sprint!").

Beyond this, however, a product owner who rejects or questions work during the sprint review might not appear to be on the same page as the rest of the Scrum team. That disconnect could come across to the stakeholders as the old, adversarial,

us-versus-them problem. The product owner and development team are on the same Scrum team and should come across as one unified team during the review meeting.

Prepare for the Demonstration

Beause all of the work the team presents at the sprint review is done (potentially shippable), it shouldn't take much preparatory work to demonstrate it. The goal is to provide transparency for inspecting and adapting the product, not to put on a glitzy Hollywood production or showcase to create excitement.

The sprint review is supposed to be an informal meeting with low ceremony and high value. Spending a lot of time to create a polished PowerPoint presentation hardly seems justified. Also, I would be concerned if I showed up at a sprint review to see working software and instead was given a PowerPoint presentation. I would be thinking, "Are these guys really done? Why won't they just show me what they created?"

Most teams have a rule of not spending more than 30 minutes to an hour per week of sprint duration to prepare for the sprint review. In addition, many also agree to show only those artifacts that were produced as a consequence of achieving the sprint goal.

Of course, there can be exceptions to the rule. I worked with an organization that developed systems under a U.S. Army contract. Most of the time, government employees (from the bureaucratic ranks) would attend the sprint reviews. Occasionally, however, the U.S. general in charge would be scheduled to attend a sprint review. In those cases, the team understandably invested a bit more time in prep and polish!

Determine Who Does What

Prior to the sprint review, the team needs to decide who on the Scrum team is going to facilitate the review and who will demonstrate the completed work. Typically the ScrumMaster facilitates, but the product owner might kick things off by welcoming members of the stakeholder community and providing a synopsis of the sprint results. As for demoing the completed work, I prefer that every member of the development team have an opportunity at some sprint review to go hands-on and demonstrate, rather than the same person always dominating the demo every sprint review. However, I try to not get too wrapped up in who is going to do these things. I let the Scrum team make that determination with a goal of maximizing the benefit of the review activity.

Approach

Figure 21.3 illustrates the sprint review activity.

The inputs to the sprint review are the sprint backlog and/or sprint goal and the potentially shippable product increment that the team actually produced.

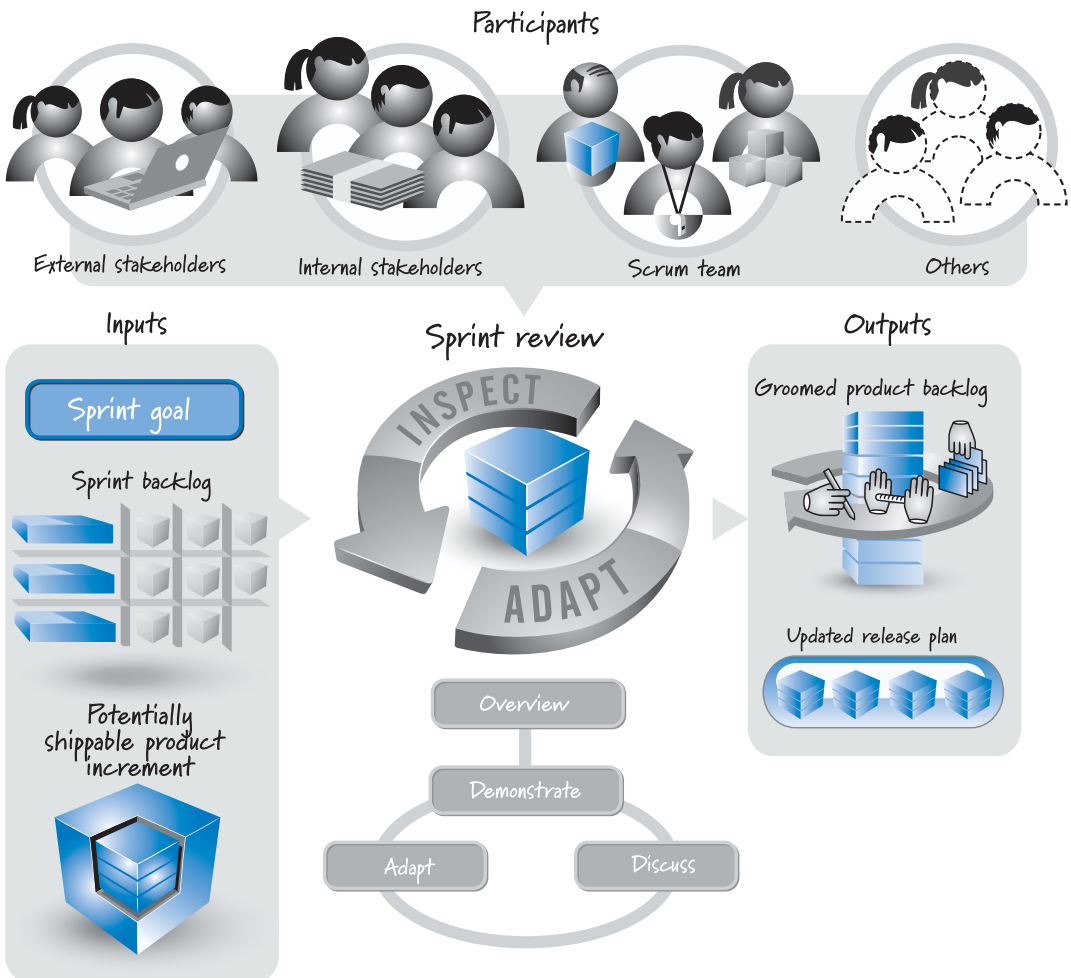


FIGURE 21.3 Sprint review activity

The outputs of the sprint review are a groomed product backlog and an updated release plan.

A common approach to conducting the sprint review includes providing a summary or synopsis of what has and has not been accomplished with regard to the sprint goal, demonstrating the potentially shippable product increment, discussing the current state of the product, and adapting the future product direction.

Summarize

The sprint review kicks off with a Scrum team member (frequently the product owner) presenting the sprint goal, the product backlog items associated with the sprint goal, and an overview of the product increment that was actually achieved

during the sprint. This information provides a summary or synopsis of how the sprint results compare with the sprint goal.

If the results don't match, the Scrum team provides an explanation. It is important that the sprint review be a blame-free environment. If the goal wasn't met, everyone participating should refrain from trying to assess blame. The purpose of the review is to describe what was accomplished and then to use the information to determine the best course of action for moving forward.

Demonstrate

The sprint review is frequently mislabeled the "**sprint demo**" or just "the demo." Although a demonstration is quite helpful in the sprint review, the demo is not the aim of the sprint review.

The most important aspect of the sprint review is in-depth conversation and collaboration among the participants to enable productive adaptations to surface and be exploited. The demonstration of what actually got built is simply a very efficient way to energize that conversation around something concrete. Nothing provides focus to the conversation like being able to actually see how something works.

As determined in the prework, one or more Scrum team members will demonstrate all relevant aspects of the product increment that was built during the sprint. In certain organizations, such as game studios, it can be even more effective to let the stakeholders actually give themselves the demo, perhaps by playing the increment of the game that was developed during the sprint.

But what if there is nothing to demo? If the team didn't get anything done and there is truly nothing to show, the sprint review will likely focus on why nothing got done and how the future work will be affected by the lack of progress during this sprint. If, on the other hand, what was built can't easily be demoed, we have a different issue. Suppose, for instance, that the team did only architectural development work this sprint (built "glue code"). In that case, the development team might argue that demonstrating glue code doesn't make any sense or isn't practical. This statement, however, is almost never true. Here's why.

For the team to work exclusively on "glue code," it would have needed to convince the product owner to allow only technical product backlog items into the sprint. As I discussed in Chapter 5, if the product owner allows such items, he must understand the value of doing the work and also must know how to determine if the work is done. Also, most teams will include in their definition of ready that the Scrum team understands how to demonstrate the item at the sprint review.

At a minimum, the team must have some set of tests to demonstrate that the work is done to the satisfaction of the product owner. Those tests must have passed because the team can show only completed work at the sprint review. So, at the very least, the team can use those tests to demonstrate at the sprint review. Usually, though, if the team members give it some thought, they can do much better. The fact that something is hard to demo is not a valid excuse to exclude it from the demo.

Discuss

Demonstrating the product increment becomes the focal point for having an in-depth conversation. Observation, comments, and reasonable discussion regarding the product and direction are strongly encouraged among the participants. The sprint review, however, is not the place for deep problem solving; that type of work should be deferred to another meeting.

Vigorous discussion allows participants who aren't on the Scrum team to ask questions, understand the current state of the product, and help guide its direction. At the same time, the Scrum team members gain a deeper appreciation for the business and marketing side of their product by getting feedback on the convergence of the product toward delighted customers or users.

Adapt

Through demonstration and discussion, the team is able to ask and answer questions, including the following:

- Do the stakeholders like what they see?
- Do they want to see changes?
- Is what we're building still a good idea in the marketplace or to our internal customers?
- Are we missing an important feature?
- Are we overdeveloping/investing in a feature where we don't have to?

Asking and answering these questions provides input on how to adapt the product backlog and release plans.

I described in Chapter 6 how most teams naturally do some grooming as part of the sprint review. As everyone involved gains a better understanding of the current development effort and where it is going, new PBIs are often created or existing PBIs are reprioritized or deleted if they are no longer needed. This grooming might affect what the team will work on in the next sprint.

Also, as I described in Chapter 18, the grooming that happens during sprint review might also affect the larger-scope release plans. For example, based on the discussion and conclusions of the sprint review, we might decide to alter one of the key release-planning variables: scope, date, or budget. Perhaps, for instance, by reviewing the current product increment we decide to stop working on a major feature of the product (change the scope). This decision will necessarily affect the current release plan.

The sprint review gives us an opportunity to identify ways to adapt, to respond to change, when it is still affordable to do so—at the end of every single sprint.

Sprint Review Issues

Sprint reviews, however, are not without issues. Having worked with many organizations using Scrum, I have noticed several common sprint review issues, including those related to sign-offs, lack of attendance, and large projects.

Sign-offs

Sign-offs can be problematic in sprint reviews. The first question to ask is whether sprint reviews are the proper venue to sign off on (approve) product backlog items. As I mentioned previously, before the sprint review even begins, the product owner must review the work to determine if it is done (meets the agreed-upon definition of done). The sprint review, therefore, should not be a formal approval or sign-off event; instead, the product backlog items should have already been "approved" by the product owner before the sprint review starts.

Let's say, however, that during the sprint review a senior-level stakeholder disagrees—he believes the product backlog item is not done. While that feedback is valuable, I would still say that if the product owner declared the original work done, it is done. In Chapter 9 I discussed how the product owner has to be the empowered central point of product leadership. For this to be true, the product owner must be in a position to definitively approve or reject work and can't have that authority usurped by a sprint review participant—no matter how senior.

That doesn't mean that the product owner should ignore comments about a feature not meeting stakeholder expectations. When this occurs, the proper course of action is to schedule a change to the feature by creating a new product backlog item to reflect the desired behavior requested by the senior stakeholder and to insert that item into the product backlog to be worked on in a future sprint. The product owner should also investigate to determine why he disconnected from the stakeholders regarding this story and make adjustments to prevent future misunderstandings.

Sporadic Attendance

The sprint review needs to be viewed as a critical inspect-and-adapt activity, one that is worth people's time to attend. Still, some organizations suffer from sporadic attendance.

One of the more common causes of sporadic attendance is that stakeholders have so much on their plates that other "higher-priority" commitments prevent them from attending sprint reviews. This is a strong indicator of organizational dysfunction—having so much concurrent work that stakeholders can't meet all of their commitments. In such situations I recommend that organizations stop working on the lower-priority products until such time as they are important enough for stakeholders to attend sprint reviews. If that day never arrives, those low-priority products, relative to the other products in the portfolio, are simply never valuable enough to work on.

Sometimes sporadic attendance is the result of people not believing that the Scrum team can produce anything worth reviewing in a few weeks' time. This is especially true when the organization first starts using Scrum. Stakeholders are accustomed to much longer periods between reviews, and the reviews they have attended heretofore might have been disappointing.

The best way to address this issue is to actually build a business-valuable, potentially shippable product increment every sprint. When teams do this, most people realize that these frequent reviews are worth their time and allow them to give fast feedback that the Scrum team can actually use.

Large Development Efforts

If you have a larger development effort with multiple Scrum teams, it might make sense to consider doing a joint sprint review. This is simply a review that includes the work completed by multiple highly interrelated teams.

There are several benefits to this approach. First, the stakeholders have to attend only one sprint review instead of several. Second, if the work was supposed to be integrated, it would make sense for the review to focus on the integrated work, not a collection of stand-alone increments. To achieve this goal, all teams need to make sure their definitions of done include integration testing, as they probably should anyway.

The downside to holding a joint sprint review meeting with more than one team is that it will probably take a bit longer and might require a larger room than any one team would have needed.

Closing

In this chapter I emphasized the purpose of the sprint review as a critical feedback loop during Scrum development. The sprint review involves a diverse set of participants, whose goal is to inspect and adapt the current product. Although the sprint review is an informal activity, the Scrum team does do minimal preparation to ensure a healthy, productive outcome. During the sprint review the Scrum team provides a synopsis of what took place and what was accomplished during the sprint. It also provides a demonstration of the product increment produced during the sprint. A vigorous discussion among the participants takes place; questions, observations, and suggestions are highly encouraged. Based on this discussion, the product backlog will be groomed and the release plan updated.

In the next chapter I will focus on the inspect-and-adapt activity for the process, the sprint retrospective.

This page intentionally left blank

Chapter 22
Sprint Retrospective

Scrum provides two inspect-and-adapt opportunities at the end of each sprint: the sprint review and the sprint retrospective. In the previous chapter I discussed the sprint review, where the team and stakeholders inspect the product itself. Let's now turn our attention to the sprint retrospective, where the Scrum team examines the process used to build that product.

I begin with an overview of the purpose of and participants in the sprint retrospective. I then describe the prework and major activities associated with a sprint retrospective, the most important of which occur after the sprint retrospective when the participants actually follow through on the improvements they identify.

Overview

In the preface to his book *Project Retrospectives*, Norm Kerth, the founder of the modern-day movement on retrospectives, summarizes the purpose of retrospectives by quoting a passage from *Winnie the Pooh* (Kerth 2001):

> *Here is Edward Bear, coming downstairs now, bump, bump, bump, bump, on the back of his head, behind Christopher Robin. It is, as far as he knows, the only way of coming downstairs, but sometimes he feels that there is another way, if only he could stop bumping for a moment and think of it.*

Sprint retrospectives give the whole Scrum team an opportunity to stop bumping along for a moment and think (see Figure 22.1). Inside the timebox of the retrospective, teams are free to examine what's happening, analyze the way they work, identify ways to improve, and make plans to implement these improvements. Anything that affects how the team creates the product is open to scrutiny and discussion, including processes, practices, communication, environment, artifacts, tools, and so on.

The sprint retrospective is one of the most important and least appreciated practices in the Scrum framework. It is important because it gives teams the chance to customize Scrum to their unique circumstances. It is underappreciated because some people have a misguided view that it takes time away from doing "real" design, build, and test work.

The sprint retrospective is a crucial contributor to the continuous improvement that Scrum offers. And while some organizations might wait to do a retrospective until the end of a large development effort, Scrum teams hold retrospectives each and